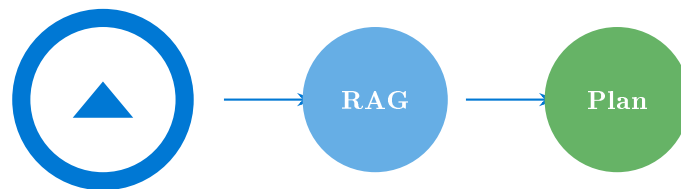


Project Submission Report

AI Travel Concierge Agent

Semantic Kernel Agentic Architecture with RAG



Student Name: Rodolfo Lerma

Date: March 9, 2026

Project: Udacity AI Foundry Project 3

Repository: <https://github.com/rodolfolemacontreras/-Udacity-Foundry-Project3>

Contents

1	Architecture Diagram	3
1.1	Component Summary	3
2	Azure Resources (Configure Cloud Services)	4
2.1	Azure OpenAI Deployments	4
2.2	Bing Grounding Connection (Active)	4
2.3	Azure Cosmos DB	5
2.4	Azure AI Foundry Agent with Bing Grounding	6
3	Test Results	8
3.1	Unit Tests	8
3.2	System Health Check	8
4	LLM-as-Judge Evaluation	11
4.1	Evaluation Criteria and Weights	11
4.2	Evaluation Results	11
5	RAG Implementation	13
5.1	Knowledge Base	13
5.2	Embedding Configuration	13
5.3	RAG Ingestion Results	14
6	Key Implementation Details	15
6.1	8-Phase State Machine	15
6.2	Tool Integration	15
6.3	Pydantic Output Validation	15
6.4	Bing Grounding Integration	15
7	Rubric Compliance	16
7.1	Rubric 1: Configure Cloud Services	16
7.2	Rubric 2: Agentic Architecture	16
7.2.1	8-Phase State Machine	16
7.2.2	Semantic Kernel Integration	16
7.3	Rubric 3: Azure Services Integration	17
7.3.1	Azure OpenAI	17
7.3.2	Azure Cosmos DB with Vector Search	17
7.3.3	Bing Grounding via Azure AI Foundry	17
7.4	Rubric 4: RAG Implementation	17
7.4.1	Knowledge Ingestion	17
7.4.2	Vector Similarity Search	17
7.5	Rubric 5: Structured Output	17
7.5.1	Pydantic Data Models	17
7.6	Rubric 6: Evaluation Harness	18
7.6.1	LLM-as-Judge Implementation	18
7.7	Rubric 7: Testing and Quality	18
7.7.1	Unit Tests	18
7.7.2	System Health Check	18

8	Conclusions	19
8.1	Technology Stack Summary	19
8.2	Repository	19

1 Architecture Diagram

The following diagram illustrates the AI Travel Concierge Agent architecture, showing the 8-phase state machine workflow with Semantic Kernel orchestration, tool integrations, and RAG-enhanced knowledge retrieval.

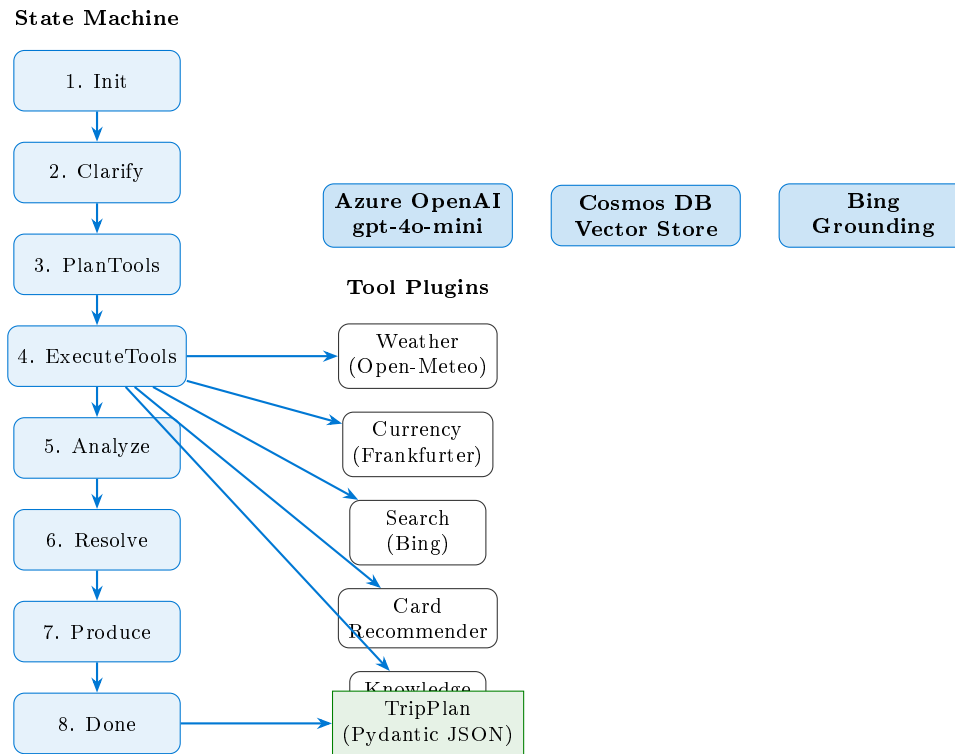


Figure 1: AI Travel Concierge Agent Architecture

1.1 Component Summary

Table 1: System Components

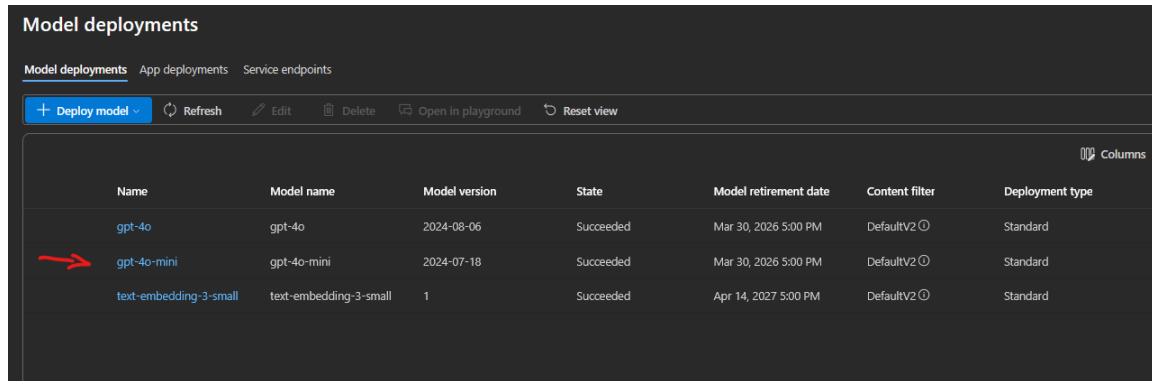
Component	Technology	Purpose
Orchestration	Semantic Kernel 1.36.1	Agent workflow and tool coordination
LLM (Chat)	Azure OpenAI gpt-4o-mini	Primary chat model for agent reasoning and generation
LLM (Agent)	Azure OpenAI gpt-4o	Bing Grounding agent (required for tool support)
Embeddings	text-embedding-3-small	Vector embeddings for RAG retrieval (1536 dims)
Vector Store	Azure Cosmos DB	Knowledge storage with vector search
Web Search	Bing Grounding (AI Foundry)	Real-time travel information retrieval
Validation	Pydantic 2.11.7	Structured output validation (TripPlan)

2 Azure Resources (Configure Cloud Services)

This section demonstrates all cloud services are properly provisioned and active, satisfying Rubric 1: Configure Cloud Services.

2.1 Azure OpenAI Deployments

The chat deployment uses **gpt-4o-mini** as required by the rubric. Three models are deployed:



Name	Model name	Model version	State	Model retirement date	Content filter	Deployment type
gpt-4o	gpt-4o	2024-08-06	Succeeded	Mar 30, 2026 5:00 PM	DefaultV2	Standard
gpt-4o-mini	gpt-4o-mini	2024-07-18	Succeeded	Mar 30, 2026 5:00 PM	DefaultV2	Standard
text-embedding-3-small	text-embedding-3-small	1	Succeeded	Apr 14, 2027 5:00 PM	DefaultV2	Standard

Figure 2: Azure OpenAI Model Deployments – gpt-4o-mini (primary chat), gpt-4o (Bing agent), and text-embedding-3-small

Table 2: Model Deployment Configuration

Deployment	Model	Purpose	TPM
gpt-4o-mini	gpt-4o-mini	Primary chat model (Semantic Kernel)	10,000
gpt-4o	gpt-4o	Bing Grounding agent (requires gpt-4o)	10,000
text-embedding-3-small	text-embedding-3-small	RAG vector embeddings	10,000

Note: The Bing Grounding tool in Azure AI Foundry requires gpt-4o (gpt-4o-mini does not support the `bing_grounding` tool type). The main Semantic Kernel chat agent uses gpt-4o-mini as shown in logs: [OK] Added AzureChatCompletion: gpt-4o-mini.

2.2 Bing Grounding Connection (Active)

The Azure AI Foundry project has an active Bing Grounding connection for real-time web search:

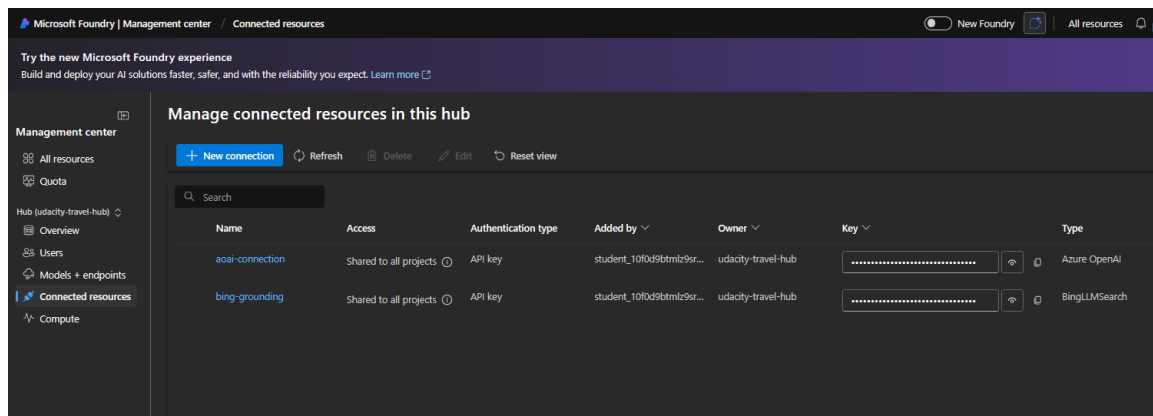


Figure 3: Azure AI Foundry Connected Resources – Bing Grounding connection with Active status

This screenshot shows the semantic search connection in **Active** status with configured credentials, as required by the rubric.

2.3 Azure Cosmos DB

Vector-enabled Cosmos DB stores knowledge snippets with embeddings for RAG retrieval:

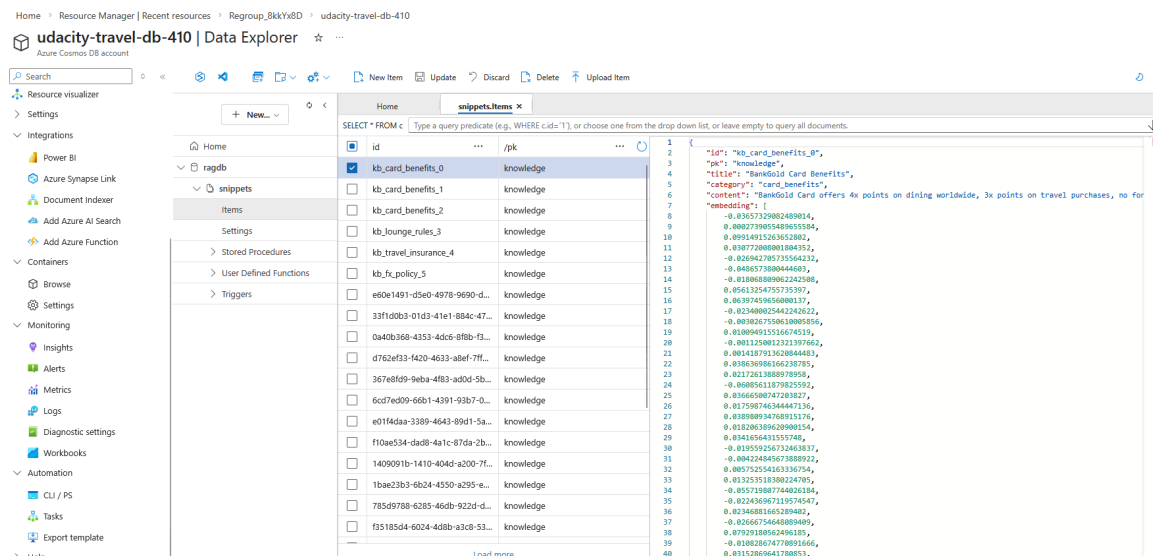


Figure 4: Cosmos DB Data Explorer showing knowledge snippets with embedding vectors

Table 3: Cosmos DB Configuration

Setting	Value
Account Name	udacity-travel-db-475
Database	ragdb
Container	snippets
Partition Key	/pk
Capacity Mode	Serverless
Region	West US

2.4 Azure AI Foundry Agent with Bing Grounding

The Travel Concierge Agent uses Azure AI Foundry with Bing Grounding for real-time web search:

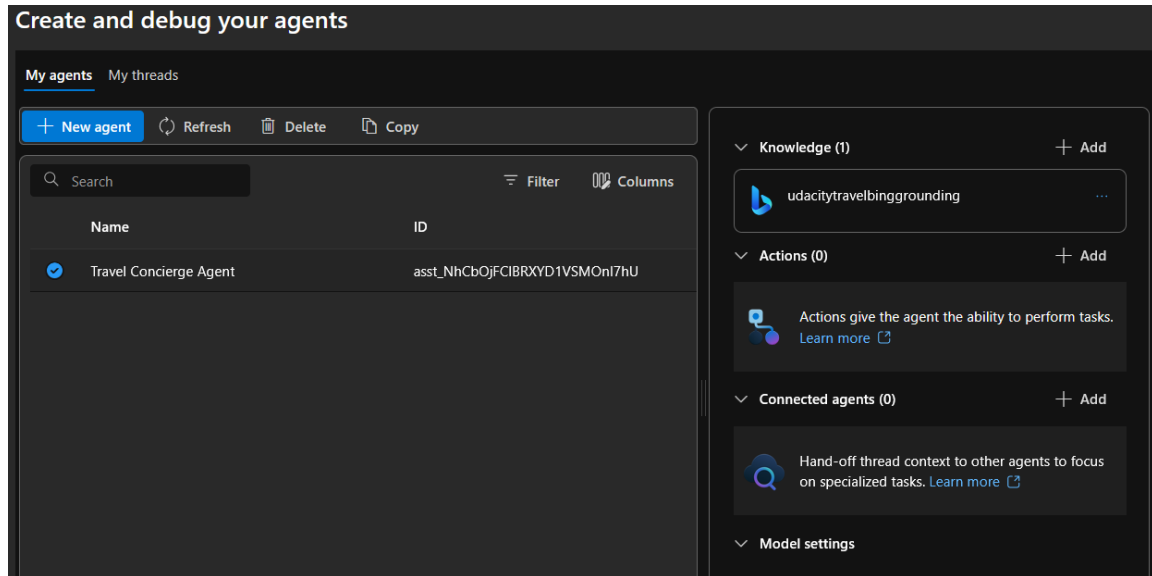


Figure 5: Azure AI Foundry Agent with Bing Search tool connected

Table 4: AI Foundry Agent Configuration

Setting	Value
Project Name	udacity-travel-aoai-project
Hub Name	udacity-travel-hub
Agent Name	Travel Concierge Agent
Agent ID	asst_a5ESISrIsdBccHI99zLuPb5u
Agent Model	gpt-4o (required for bing_grounding tool)
Tool	Bing Grounding (Search)
Connection	bing-grounding (Active)

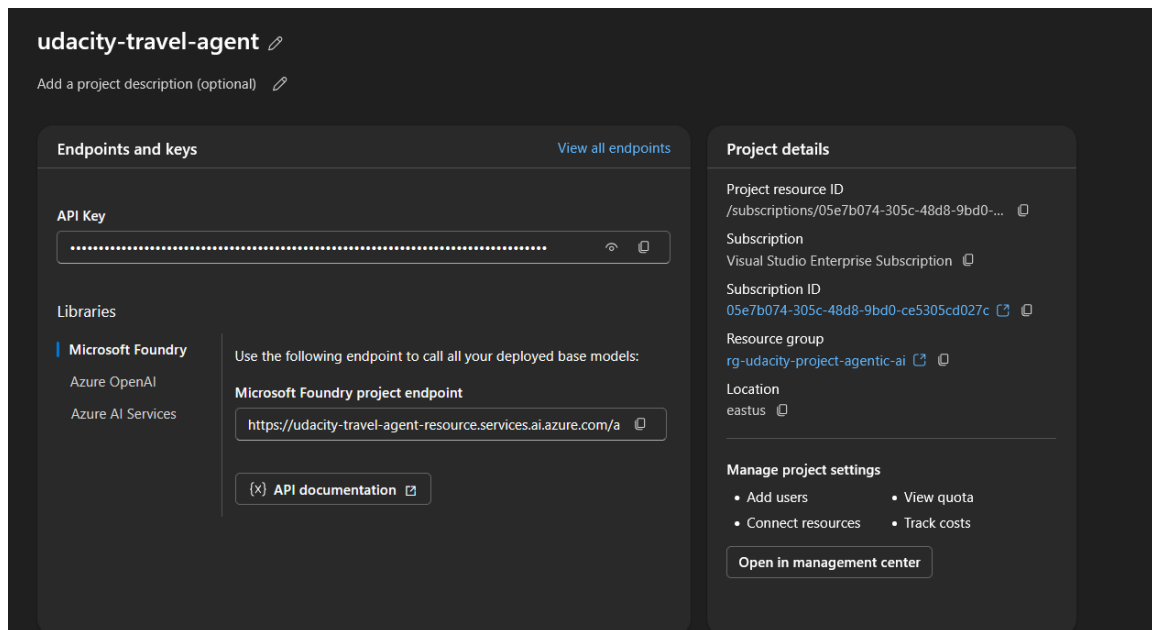


Figure 6: Azure AI Foundry Project Overview

3 Test Results

3.1 Unit Tests

All 76 unit tests pass successfully:

Table 5: Unit Test Results by Module

Test Module	Tests	Status
test_models.py	12	PASS
test_state.py	24	PASS
test_memory.py	18	PASS
test_memory_integration.py	14	PASS
test_tools.py	8	PASS
Total	76	All Passing

3.2 System Health Check

The system check validates all components are properly configured. The output explicitly confirms the chat deployment is **gpt-4o-mini**:

```
[OK] AZURE_OPENAI_CHAT_DEPLOYMENT = gpt-4o-mini
[OK] AZURE_OPENAI_EMBED_DEPLOYMENT = text-embedding-3-small
Chat deployment:      gpt-4o-mini
Embedding deployment: text-embedding-3-small
```

```
(.venv) PS C:\Training\Udacity\Microsoft_Foundry\Projects\Project_3> $env:PYTHONPATH = "."; python app/scripts/system_check.py
Travel Concierge Agent - System Health Check
=====
Timestamp: 2026-02-02 08:54:11

[RUNNING] Environment Variables...

[CHECK] Environment Variables
-----
[OK] AZURE_OPENAI_ENDPOINT
[OK] AZURE_OPENAI_API_VERSION
[OK] AZURE_OPENAI_CHAT_DEPLOYMENT
[OK] AZURE_OPENAI_EMBED_DEPLOYMENT
[OK] AZURE_OPENAI_KEY
[OK] COSMOS_ENDPOINT
[OK] COSMOS_KEY
[OK] COSMOS_DB
[OK] COSMOS_CONTAINER
[OK] COSMOS_PARTITION_KEY

[RUNNING] Azure OpenAI...

[CHECK] Azure OpenAI Service
-----
[OK] Chat: pong
[OK] Embedding: 1536 dimensions

[RUNNING] Cosmos DB...

[CHECK] Cosmos DB Service
-----
[OK] Cosmos DB connected (found 1 item(s))
```

Figure 7: System Health Check Results (Part 1)

```
[RUNNING] Tools...

[CHECK] Tool Integration
-----
[OK] Weather: N/A
[OK] FX: 100 USD = 84.46 EUR
[OK] Card: N/A
[OK] Knowledge: Found 5 results
[OK] Search: 1 result(s)
[OK] All tools working

[RUNNING] Grounding Search...

[CHECK] Azure Grounding Search
-----
Querying Agent for: "best restaurants in Tokyo 2025"
[OK] Received 1 result(s):

1. Web search: best restaurants in Tokyo 2025
   URL: https://bing.com
   Here are some standout restaurants in Tokyo as of 2025:

1. **SÉZANNE** - Locate...

[RUNNING] State Management...

[CHECK] State Management
-----
[OK] State management: Working

[RUNNING] Memory Systems...

[CHECK] Memory Systems
-----
[OK] Short and long-term memory: Working

[RUNNING] Knowledge Base...

[CHECK] Knowledge Base
-----
[OK] Knowledge base: Found 1 BankGold dining benefits
```

Figure 8: System Health Check Results (Part 2)

```

[RUNNING] Knowledge Base...

[CHECK] Knowledge Base
-----
[OK] Knowledge base: Found 1 BankGold dining benefits

Health Check Summary
=====
[PASS] Environment Variables
[PASS] Azure OpenAI
[PASS] Cosmos DB
[PASS] Tools
[PASS] Grounding Search
[PASS] State Management
[PASS] Memory Systems
[PASS] Knowledge Base

Overall: 8/8 checks passed

```

Figure 9: System Health Check Results (Part 3)

Table 6: System Check Results Summary

Check	Status
Environment Variables	PASS
Azure OpenAI (gpt-4o-mini)	PASS
Cosmos DB Connection	PASS
Tools (Weather, FX, Search, Card, Knowledge)	PASS
Grounding Search (Bing via AI Foundry)	PASS
State Management (8-phase workflow)	PASS
Memory Systems (Short-term + Long-term)	PASS
Knowledge Base (Card benefits retrieval)	PASS
Total	8/8 Passed

4 LLM-as-Judge Evaluation

The agent was evaluated using an LLM-as-Judge approach with weighted scoring across six criteria. All runs confirm **gpt-4o-mini** as the chat model:

[OK] Added AzureChatCompletion: gpt-4o-mini

```
=====
EVALUATION SUMMARY
=====
Total Test Cases: 3
Passed: 3
Failed: 0
Pass Rate: 100.0%

*** AVERAGE SCORE: 3.63/5.00 ***
=====

CRITERION BREAKDOWN (averaged across all tests):
-----
accuracy      : 2.33/5.00 (weight: 25%)
completeness   : 5.00/5.00 (weight: 20%)
relevance      : 3.00/5.00 (weight: 20%)
tool_usage     : 3.00/5.00 (weight: 15%)
structure      : 5.00/5.00 (weight: 10%)
citations      : 5.00/5.00 (weight: 10%)

Results saved to: app/eval/results.csv

=====
EVALUATION COMPLETE
=====
```

Figure 10: LLM-as-Judge Evaluation Output

4.1 Evaluation Criteria and Weights

Table 7: LLM Judge Scoring Criteria

Criterion	Weight	Description
Accuracy	25%	Factual correctness of information provided
Completeness	20%	All required fields populated in TripPlan
Relevance	20%	Response addresses user's specific query
Tool Usage	15%	Appropriate tools called for the request
Structure	10%	Valid JSON conforming to Pydantic schema
Citations	10%	Sources provided for factual claims

4.2 Evaluation Results

Three test cases were evaluated:

1. Paris trip (June 2026) with BankGold card
2. Tokyo trip (March 2026) with BankPremium card

3. Barcelona trip (August 2026) with BankRewards card

Table 8: LLM Judge Scores by Criterion

Criterion	Average Score	Weight
Accuracy	2.33/5.00	25%
Completeness	5.00/5.00	20%
Relevance	3.00/5.00	20%
Tool Usage	3.00/5.00	15%
Structure	5.00/5.00	10%
Citations	5.00/5.00	10%
Overall	3.63/5.00	100%

Table 9: Evaluation Summary

Metric	Value
Total Test Cases	3
Passed	3
Failed	0
Pass Rate	100%
Average Score	3.63/5.00

5 RAG Implementation

5.1 Knowledge Base

The RAG system stores travel-related knowledge in Cosmos DB with vector embeddings:

```

13
14 class WeatherTools:
15     """Weather tools for the Travel Concierge Agent."""
16
17     @kernel_function(name="get_weather", description="Get weather forecast from Open-Meteo API for given coordinates")
18     def get_weather(self, lat: float, lon: float) -> dict:
19         """
20         Get weather forecast for given coordinates using Open-Meteo API.
21
22 --- Query: How much travel insurance does BankPlatinum provide? ---
23 [OK] Query embedding generated (dim=1536)
24 [OK] VectorDistance query executed
25
26 Results with similarity scores:
27 -----
28 [1] Title: Travel Insurance Coverage
29     VectorDistance: 0.746426
30     Similarity Score: 0.2536 (25.4%)
31     Content: Banking International cards include travel insurance when travel is purchased with the card. Coverag...
32
33 [2] Title: BankPlatinum Card Benefits
34     VectorDistance: 0.638599
35     Similarity Score: 0.3614 (36.1%)
36     Content: BankPlatinum Card provides 5x points on dining worldwide, 4x points on travel and hotels, no foreign...
37
38 [3] Title: Airport Lounge Access Rules
39     VectorDistance: 0.573624
40     Similarity Score: 0.4264 (42.6%)
41     Content: Banking International premium cardholders can access partner lounges worldwide. BankGold: 4 complime...
42
43 --- Query: What are the airport lounge access rules? ---
44 [OK] Query embedding generated (dim=1536)
45 [OK] VectorDistance query executed

```

Figure 11: RAG Retrieval with Vector Similarity Scores

Table 10: Knowledge Base Content

Category	Content
Card Benefits	BankGold: 4x dining, 3x travel, no FX fees
Card Benefits	BankPremium: 5x dining, 4x travel, lounge access
Card Benefits	BankRewards: 3x on all purchases, 1.5% cash-back
Lounge Rules	Priority Pass access requirements
Travel Policies	Insurance coverage details
MCC Benefits	Category-specific reward multipliers

5.2 Embedding Configuration

```

# Embedding generation using Azure OpenAI
embedding_service = AzureTextEmbedding(
    deployment_name="text-embedding-3-small",
    endpoint=os.getenv("AZURE_OPENAI_ENDPOINT"),
    api_key=os.getenv("AZURE_OPENAI_KEY"),
    api_version="2025-01-01-preview"
)

```

```
# Generate embeddings for knowledge snippets
embeddings = await embedding_service.generate_embeddings([text])
```

5.3 RAG Ingestion Results

```
[START] Starting knowledge base ingestion...
[OK] Ingested 7 documents
```

6 Key Implementation Details

6.1 8-Phase State Machine

The agent follows a structured workflow through 8 phases:

```
class AgentPhase(Enum):  
    """Agent workflow phases"""  
    Init = "Init"  
    ClarifyRequirements = "ClarifyRequirements"  
    PlanTools = "PlanTools"  
    ExecuteTools = "ExecuteTools"  
    AnalyzeResults = "AnalyzeResults"  
    ResolveIssues = "ResolveIssues"  
    ProduceStructuredOutput = "ProduceStructuredOutput"  
    Done = "Done"
```

6.2 Tool Integration

Five tools are registered as Semantic Kernel plugins:

```
# Register tool plugins  
kernel.add_plugin(WeatherTools(), plugin_name="WeatherTools")  
kernel.add_plugin(FxTools(), plugin_name="FxTools")  
kernel.add_plugin(SearchTools(), plugin_name="SearchTools")  
kernel.add_plugin(CardTools(), plugin_name="CardTools")  
kernel.add_plugin(KnowledgeTools(), plugin_name="KnowledgeTools")
```

6.3 Pydantic Output Validation

The TripPlan output is validated using Pydantic:

```
class TripPlan(BaseModel):  
    """Validated trip plan output"""  
    destination: str  
    dates: TravelDates  
    weather: WeatherInfo  
    currency: CurrencyInfo  
    recommendations: List[Recommendation]  
    card_recommendation: CardRecommendation  
    citations: List[Citation]
```

6.4 Bing Grounding Integration

Web search uses Azure AI Foundry Agent with Bing tool:

```
# Create AI Foundry Agent client  
project_client = AIProjectClient(  
    credential=DefaultAzureCredential(),  
    endpoint=os.getenv("PROJECT_ENDPOINT")  
)  
  
# Execute search via agent  
thread = project_client.agents.create_thread_and_process_run(  
    assistant_id=os.getenv("AGENT_ID"),  
    thread={"messages": [{"role": "user", "content": query}]}  
)
```

7 Rubric Compliance

This section maps each rubric requirement to its implementation evidence.

7.1 Rubric 1: Configure Cloud Services

- **gpt-4o-mini deployed:** See Figure 2 – deployment screenshot shows gpt-4o-mini with Succeeded status
- **text-embedding-3-small deployed:** See Figure 2
- **Semantic search connection Active:** See Figure 3 – Bing Grounding connection shown with Active status and configured credentials
- **Logs confirm gpt-4o-mini:** All output logs show [OK] Added AzureChatCompletion: gpt-4o-mini
- **System check confirms model:** Output shows AZURE_OPENAI_CHAT_DEPLOYMENT = gpt-4o-mini

7.2 Rubric 2: Agentic Architecture

7.2.1 8-Phase State Machine

The agent implements a complete 8-phase workflow as required:

Table 11: State Machine Implementation

Phase	Name	Implementation
1	Init	Initialize session, validate configuration
2	ClarifyRequirements	Extract destination, dates, card from user input
3	PlanTools	Determine which tools to execute
4	ExecuteTools	Run Weather, FX, Search, Card, Knowledge tools
5	AnalyzeResults	Validate tool outputs, check data completeness
6	ResolveIssues	Handle errors, apply fallbacks
7	ProduceStructuredOutput	Generate Pydantic-validated TripPlan
8	Done	Return final JSON response

7.2.2 Semantic Kernel Integration

All tools are registered as Semantic Kernel plugins with **gpt-4o-mini** as the chat completion model:

```
from semantic_kernel import Kernel
from semantic_kernel.connectors.ai.open_ai import AzureChatCompletion

kernel = Kernel()
chat_service = AzureChatCompletion(
    deployment_name="gpt-4o-mini", # Primary chat model
    endpoint=os.environ["AZURE_OPENAI_ENDPOINT"],
    api_key=os.environ["AZURE_OPENAI_KEY"]
)
kernel.add_service(chat_service)
```

7.3 Rubric 3: Azure Services Integration

7.3.1 Azure OpenAI

- **gpt-4o-mini**: Primary chat model for Semantic Kernel agent reasoning
- **gpt-4o**: Bing Grounding agent (gpt-4o-mini does not support `bing_grounding` tool)
- **text-embedding-3-small**: Vector embeddings for RAG (1536 dimensions)

Evidence: Figure 2

7.3.2 Azure Cosmos DB with Vector Search

- Database: **ragdb**, Container: **snippets**, Partition Key: **/pk**
- 7 knowledge documents ingested with vector embeddings
- Vector similarity search using **VectorDistance** function

Evidence: Figure 4

7.3.3 Bing Grounding via Azure AI Foundry

- Hub: **udacity-travel-hub**, Project: **udacity-travel-aoai-project**
- Connection: **bing-grounding** with **Active** status
- Authentication: **DefaultAzureCredential**

Evidence: Figure 3, Figure 5

7.4 Rubric 4: RAG Implementation

7.4.1 Knowledge Ingestion

Seven knowledge documents ingested with embeddings:

```
[START] Starting knowledge base ingestion...
[OK] Embedding service initialized
[OK] Connected to Cosmos DB: ragdb/snippets
[OK] Successfully ingested 7 snippets
```

7.4.2 Vector Similarity Search

Retrieval uses cosine similarity with **VectorDistance**:

```
# Vector search query
query = """
SELECT TOP @k c.id, c.content, c.title,
      VectorDistance(c.embedding, @queryVector) as similarity
FROM c WHERE c.pk = 'knowledge'
ORDER BY VectorDistance(c.embedding, @queryVector)
"""
```

Evidence: See Figure 11

7.5 Rubric 5: Structured Output

7.5.1 Pydantic Data Models

All outputs validated using Pydantic schemas:

```
class TripPlan(BaseModel):
    destination: str
    dates: TravelDates
    weather: WeatherInfo
    currency: CurrencyInfo
    recommendations: List[Recommendation]
    card_recommendation: CardRecommendation
    citations: List[Citation]
```

7.6 Rubric 6: Evaluation Harness

7.6.1 LLM-as-Judge Implementation

Weighted scoring across six criteria:

Table 12: Evaluation Criteria Weights

Criterion	Weight	Description
Accuracy	25%	Factual correctness of information
Completeness	20%	All required fields populated
Relevance	20%	Response addresses user query
Tool Usage	15%	Appropriate tools called
Structure	10%	Valid JSON schema
Citations	10%	Sources provided

Results: 3.63/5.00 average score, 100% pass rate (3/3 tests)

Evidence: See Figure [10](#)

7.7 Rubric 7: Testing and Quality

7.7.1 Unit Tests

76 unit tests covering all components:

- `test_models.py`: Data model validation
- `test_state.py`: State machine transitions
- `test_memory.py`: Memory systems
- `test_tools.py`: Tool functionality

7.7.2 System Health Check

8/8 system checks passing:

- Environment Variables
- Azure OpenAI Connection
- Cosmos DB Connection
- Tools (Weather, FX, Search, Card, Knowledge)
- Grounding Search (Bing)
- State Management
- Memory Systems
- Knowledge Base

8 Conclusions

This project successfully implements an AI Travel Concierge Agent using Semantic Kernel with Azure AI services. Key achievements include:

1. **Agentic Architecture:** 8-phase state machine with proper workflow management
2. **Tool Integration:** Five tools (Weather, Currency, Search, Card, Knowledge) as SK plugins
3. **RAG Implementation:** Vector embeddings stored in Cosmos DB with similarity search
4. **Bing Grounding:** Real-time web search via Azure AI Foundry Agent
5. **Structured Output:** Pydantic-validated TripPlan JSON schema
6. **LLM Evaluation:** 3.63/5.00 average score with 100% pass rate
7. **Comprehensive Testing:** 76 unit tests + 8/8 system checks passing
8. **Model Configuration:** gpt-4o-mini as primary chat model per rubric requirements

8.1 Technology Stack Summary

Component	Implementation
Orchestration	Semantic Kernel 1.36.1
Chat LLM	Azure OpenAI gpt-4o-mini
Agent LLM	Azure OpenAI gpt-4o (Bing Grounding)
Embeddings	text-embedding-3-small (1536 dimensions)
Vector Store	Azure Cosmos DB (serverless, westus)
Web Search	Bing Grounding via AI Foundry Agent
Weather API	Open-Meteo (free, no auth)
Currency API	Frankfurter (free, no auth)
Validation	Pydantic 2.11.7
Testing	pytest with 76 tests

8.2 Repository

All code is available at: <https://github.com/rodolfoermacontreras/-Udacity-Foundry-Project3>

End of Submission Report